

L'IA pour gérer la dette technique

À propos de moi

- Directeur de l'ingénierie logicielle de Koddian chez Kumojin
- Artisan logiciel
- 15 ans d'expérience
- Utilisateur avancé de l'IA pour développer depuis 18 mois



Le problème : la dette technique s'accumule

- Dégradation de l'écosystème externe
- Accumulation des décisions internes
- Complexité de la croissance

L'IA peut aider... ou aggraver

Potentiel

- Analyser des milliers de lignes
- Détecter des patterns obsolètes
- Proposer des migrations
- Automatiser le refactoring

Risques

- Suggestions hors contexte
- Estimations irréalistes
- Changements non testés
- **La dette technique augmente au lieu de diminuer**

Définitions : les outils

LLMs (Large Language Models)

Systèmes d'IA générative avancés qui comprennent et génèrent du langage naturel.

Exemples : GPT, Claude, Gemini

Agent de codage

Outil qui lit, modifie ou corrige du code de manière autonome en utilisant le langage naturel comme entrée.

Exemples : Codex, Claude Code, Cursor

Comment fonctionne un LLM ?

Trois grandes étapes pour créer un LLM :

1. Pré-entraînement

Le modèle apprend à prédire le prochain token à partir de milliards d'exemples de texte.

💰 Millions de \$ · ⌚ Semaines/mois · 🖥️ Milliers de GPU

2. Fine-tuning supervisé (SFT)

Le modèle apprend à suivre des instructions à partir d'exemples annotés par des humains.

📊 Moins de données · 👤 Exemples sélectionnés · 🎯 Suivi d'instructions

3. Apprentissage par renforcement avec feedback humain (RLHF)

Le modèle apprend à être utile, honnête et inoffensif via les préférences humaines.

👤 Feedback humain · 🏆 Reward model · ✅ Alignement

Que se passe-t-il quand vous envoyez un prompt ?

1. Votre prompt est **tokenisé** (divisé en morceaux)

```
"Analyse cette fonction" → ["Analyse", "cette", "fonction"]
```

2. Le modèle traite ces tokens à travers son réseau de neurones
3. **Prédit le token suivant LE PLUS PROBABLE** basé sur les patterns appris

```
"Analyse cette" → next token: probablement "fonction"
```

4. Ajoute ce token à la séquence
5. Répète jusqu'à générer une réponse complète

⚠ **Point clé** : "Le plus probable" ≠ "Le correct"

C'est de la **prédiction statistique**, pas de la magie.

Capacités clés

✓ Analyse massive

- Milliers de lignes en quelques secondes

✓ Reconnaissance de patterns

- Anti-patterns, code smells

✓ Apprentissage par l'exemple

- Donne-lui des exemples, il adapte

✓ Connexion outils

- Lire fichiers, exécuter code

✓ Génération de code

- Boilerplate, tests, refactors guidés

Limitations

⚠ Knowledge cutoff (dépend des modèles)

- Ne connaît pas les versions récentes des librairies par exemple

⚠ Hallucinations

- Peut inventer des APIs

⚠ Context window (limité selon le modèle)

- Limite de mémoire

⚠ Raisonnement complexe

- Algorithmes, maths avancées

⚠ Non-déterminisme

- Même prompt → réponses différentes

La philosophie

Deux approches pour utiliser l'IA dans le développement

Vibe Coding

Prompts	"Crée une page de profil utilisateur"
Validation	Pas de review
Compréhension	Copy-paste aveugle
Rapidité	 Rapide au début
Long terme	 Dette technique

Mentalité: "Ça marche, c'est bon"

Ingénierie assistée par IA

Prompts	"Refactor UserService pour utiliser DI"
Validation	Review + tests à chaque étape
Compréhension	Comprendre et adapter
Rapidité	 Plus lent au début
Long terme	 Maintenable

Mentalité: "L'IA propose, je valide"

Objectifs de cette présentation

Ce que vous allez voir

-  Faire de l'**ingénierie assistée par IA** (pas du vibe coding)
-  Comprendre et **valider chaque étape**
-  Appliquer sur un **vrai code legacy**

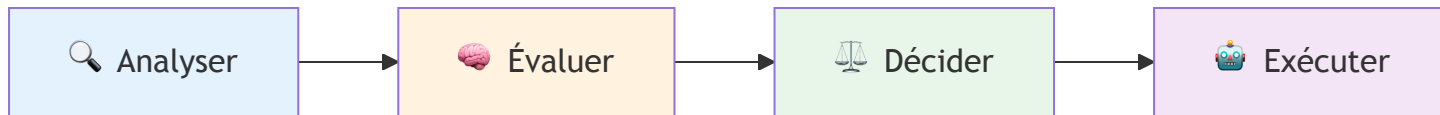
Notre cobaye : extreme-carpaccio

Un kata qui illustre une **situation professionnelle courante** :

- Hériter d'un projet non maintenu depuis plusieurs années.
- Malgré sa qualité initiale, le code présente aujourd'hui une dette technique significative.

Point positif : une suite de tests existante

Roadmap des démonstrations



Le pattern à observer

L'IA propose → L'humain valide → L'IA exécute

Pourquoi ce pattern ?

- Tire parti de la vitesse de l'IA sans sacrifier le contrôle
- Réduit la charge cognitive : l'IA explore, vous décidez
- Scalable : adaptable du code individuel aux systèmes complexes

Analyse du code

Prompt d'analyse de la dette technique

Analyze this codebase for technical debt and architectural issues.

Focus on:

- Code smells and anti-patterns
- Outdated dependencies and framework versions
- Architectural problems (coupling, separation of concerns, etc.)
- Security vulnerabilities from deprecated packages
- Performance bottlenecks or inefficient patterns
- Missing error handling
- Inconsistent code patterns across the codebase

For each issue you identify:

1. Specify the exact location (file and line numbers)
2. Explain why it's problematic
3. Rate the severity (critical, high, medium, low)
4. Estimate effort to fix (hours or story points)
5. Suggest a specific remediation approach

After the analysis, generate a prioritized action plan with:

- Quick wins (high value, low effort)
- Critical issues that block modernization
- Long-term refactoring goals

Analyse du code - Résultats

Ce que l'IA a trouvé

- **Dépendances obsolètes** : React 0.12, jasmine-node
- **Vulnérabilités CVE** : des failles de sécurité connues
- **Patterns anti-code** : callback hell, code dupliqué
- **Structure claire** : emplacement exact + sévérité + effort + solution

Ce qu'il faut valider

- **Contexte métier** : "exécution de code arbitraire" → oui, c'est une feature
- **Estimations** : "Migration TypeScript : 8-10 jours" → indicateur, pas vérité
- **Knowledge cutoff** : Ne connaît pas toujours les dernières versions
- **Priorités** : L'IA ne connaît pas vos contraintes projet

Migration majeure : React 19.2

Prompt de migration

Plan the React migration from the frontend to the latest version (19.2).

Constraints:

- Do not migrate to TypeScript
- Ensure libraries compatibility with React 19.2
- Propose E2E testing strategy to validate before and after

Migration majeure : React 19.2

Approche interactive

- Claude analyse et pose des questions
- Vous guidez la stratégie
- Claude génère un plan détaillé

Comparer avant de migrer

L'agent analyse les options et recommande la meilleure approche pour votre contexte.

Exemple de prompt

```
Search body-parser alternatives.
```

```
Compare:
```

- Performance and bundle size
- API compatibility
- Migration effort
- Active maintenance

```
Recommend the best option for our use case.
```


Supprimer plutôt que migrer

La meilleure dépendance, c'est celle qu'on n'a pas. L'agent identifie ce qui peut revenir à des APIs natives.

Exemple de prompt

```
Analyze our use of lodash in the codebase.
```

```
Identify functions that can be replaced with native JavaScript APIs.
```

```
For each replacement:
```

- Show the native equivalent
- Confirm identical behavior
- Highlight any edge cases
- Estimate migration effort

```
Prioritize high-usage functions first.
```

Automatiser avec un codemod sur mesure

L'IA cherche un codemod existant, et si rien ne convient, elle en écrit un avec dry-run et validation humaine.

Exemple de prompt

```
/codemod
```

```
Migrate our jasmine-node test suite to jest spy semantics:
```

- spyOn(x, y).andReturn(z) → jest.spyOn(x, y).mockReturnValue(z)
- spyOn(x, y).andCallFake(fn) → jest.spyOn(x, y).mockImplementation(fn)
- jasmine.any(Type) → expect.any(Type)

40+ occurrences sur 4 fichiers de specs. Aucun codemod public n'existe pour cette transformation.

Outil : Codemod MCP

```
npx codemod ai
```

Et dans la vraie vie ?

- Remplacer React-Scripts par Vite
- Migrer de React class components à function components
- Migrer React-Router 5x à 7x
- Migration majeure de Material UI
- Automatiser la mise à jour de dépendances
- Support pour faire des audits

Et après ? Payer la dette en continu

Jusqu'ici : un dev qui rembourse la dette de manière réactive. La suite : des agents qui paient en continu.

Le problème de drift

Les agents reproduisent les patterns existants, y compris les mauvais. Le code dérive lentement.

Chez OpenAI, **20 % de la semaine (tous les vendredis)** à **nettoyer le « AI slop »** avant qu'ils n'automatisent ça.

Source : *Harness engineering*, Ryan Lopopolo, OpenAI (fév. 2026).

gh-aw – la version open source

`github/gh-aw` : workflows agentiques dans GitHub Actions, déclarés en markdown.

```
gh extension install github/gh-aw
```

La réponse : le harness

1. Encoder les « golden principles » dans le repo
2. Des agents en tâche de fond qui scannent les dérives
3. Des PRs de cleanup petites, ciblées, automerge
4. **Payer la dette de manière continue** plutôt qu'en gros chantiers périodiques

Formaliser ce que l'agent doit savoir

Sans contexte, l'agent réplique ce qu'il voit dans le code, y compris les mauvaises pratiques.

AGENTS.md

Toujours chargé dans le contexte.

Conventions

- Tests: vitest, pas jasmine
- Logger: pino, jamais console.*
- Pas de moment.js

Skills

L'agent lit les descriptions en contexte et charge le contenu complet quand la tâche correspond.

- **Auto** : writing-tests , debugging , pr-content ...
- **Explicite** : /code-review , /codemod , /release

Outil : rulesync

Source unique → configs pour **25+ outils** : Claude Code, Cursor, Copilot, Gemini CLI, Codex, Cline... Rules, skills, commands, MCP, permissions.

```
npx rulesync generate
```

Points clés

Cinq démos, une approche

1. **Analyser** : l'IA explore, nous validons les findings
2. **Planifier** : l'IA propose un plan, nous l'orientons
3. **Décider** : l'IA compare les alternatives, nous choisissons
4. **Simplifier** : l'IA repère les dépendances inutiles, nous mesurons l'effort
5. **Automatiser** : l'IA écrit le codemod, nous validons le diff

Par où commencer sur vos projets

- **Commencez petit** : analyse de code (faible risque)
- **Développez votre esprit critique** : questionnez tout
- **Élargissez progressivement** : migration → refactoring → automatisation
- **Gardez le contrôle** : l'IA propose, vous décidez

Questions ?

<https://github.com/xballoy/presentation-ai-tech-debt>